

Resolución: Cestas de Fruta del Señor Martín mediante Backtracking

Este documento presenta la resolución detallada del problema de diseño de un algoritmo de **Backtracking** (Vuelta Atrás) para calcular el número total de cestas de frutas posibles cuyo peso total se encuentre entre los 200 gramos y 1 de kilogramo (1000 gramos), además de obtener el peso acumulado de todas las cestas válidas combinadas.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Representamos cada cesta como una tupla binaria de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa la presencia o ausencia de la fruta de tipo i en la cesta:

- $x_i = 1$: Se incluye la fruta de tipo i en la cesta.
- $x_i = 0$: No se incluye la fruta de tipo i en la cesta.

Asumimos que disponemos de un vector constante de pesos $W = [w_1, w_2, \dots, w_n]$ que indica el peso de una pieza de cada tipo de fruta i (donde $w_i \geq 50$ gramos $\forall i$).

2. Restricciones Explícitas

Las restricciones explícitas definen el dominio binario de cada variable de decisión:

$$x_i \in \{0, 1\} \quad \forall i \in \{1, 2, \dots, n\}$$

3. Restricciones Implícitas

La restricción implícita exige que el peso total de las frutas seleccionadas en la cesta esté estrictamente dentro del rango $[200, 1000]$ gramos:

$$200 \leq \sum_{i=1}^n (w_i \cdot x_i) \leq 1000$$

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se representa mediante un **árbol de decisión binario**:

- **Raíz (Nivel 0)**: Cesta vacía sin evaluar ninguna fruta.

- **Nivel i ($1 \leq i \leq n$):** Representa la toma de decisión para la fruta de tipo i .
- **Ramificación:** De cada nodo en el nivel $i - 1$ se abren dos opciones:
 - Rama izquierda: Asigna $x_i = 0$ (no incluir la fruta).
 - Rama derecha: Asigna $x_i = 1$ (incluir la fruta).
- **Hojas (Nivel n):** Cestas con una composición completa evaluada para las n frutas.

Tamaño del Árbol (Búsqueda Exhaustiva)

Sin la aplicación de podas en el árbol de búsqueda:

1. Número de hojas (soluciones candidatas evaluadas): 2^n
2. Número total de nodos en el árbol:

$$1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$$

c) Predicados Acotadores (Pruning)

Definimos P_{act} como el peso acumulado de las frutas que ya hemos decidido incluir hasta el nivel actual k :

$$P_{\text{act}} = \sum_{j=1}^{k-1} (w_j \cdot x_j)$$

Y definimos P_{rest} como la suma de los pesos de todas las frutas restantes sobre las que aún no hemos tomado una decisión (desde la fruta k hasta la n):

$$P_{\text{rest}} = \sum_{j=k}^n w_j$$

1. Poda por Exceso de Peso

Dado que los pesos de todas las frutas son estrictamente positivos ($w_i \geq 50$), añadir más elementos a la cesta nunca reducirá su peso. Si el peso actual de la cesta supera el límite de 1000 gramos, podemos abortar la exploración de esa rama de inmediato.

- Condición de poda:

$$P_{\text{act}} > 1000$$

2. Poda por Defecto de Peso

Si el peso que ya llevamos acumulado (P_{act}) sumado al peso de todas las frutas disponibles que quedan por decidir (P_{rest}) es menor que el mínimo requerido de 200 gramos, es imposible alcanzar el rango de validez aunque incluyemos todas las frutas restantes.

- Condición de Poda:

$$P_{\text{act}} + P_{\text{rest}} < 200$$

3. Predicado Acotador de Viabilidad

Un nodo en el nivel k es viable para continuar su exploración recursiva si y solo si cumple:

$$P_{\text{act}} \leq 1000 \quad \wedge \quad P_{\text{act}} + P_{\text{rest}} \geq 200$$

d) Algoritmo en Pseudocódigo y Análisis de Coste

A continuación se presenta el pseudocódigo recursivo que resuelve el problema contando las cestas válidas y acumulando su peso total.

```

Procedimiento ResolverCestasDeFruta(n, W)
Inicio
    total_cestas <- 0
    peso_total_acumulado <- 0

    // Calcular el peso de todas las frutas para la poda por defecto inicial
    suma_restante_inicial <- 0
    Para i <- 1 Hasta n Con Paso 1 Hacer
        suma_restante_inicial <- suma_restante_inicial + W[i]
    FinPara

    // Procedimiento recursivo de backtracking
    Procedimiento Backtracking(nivel, peso_actual, peso_restante)
    Inicio
        // 1. PREDICADOS ACOTADORES (PODAS)
        Si peso_actual > 1000 Entonces
            Retornar
        FinSi

        Si peso_actual + peso_restante < 200 Entonces
            Retornar
        FinSi

        // 2. CASO BASE (Se ha tomado una decisión para todas las frutas)
        Si nivel = n + 1 Entonces
            // El peso_actual está garantizado en el rango [200, 1000] por las pc
            total_cestas <- total_cestas + 1
            peso_total_acumulado <- peso_total_acumulado + peso_actual
            Retornar
        FinSi

        // 3. RAMIFICACIÓN
        // Opción A: No incluir la fruta del nivel actual (x_nivel = 0)
        // El peso acumulado no sube, pero el peso restante disponible disminuye
        Backtracking(nivel + 1, peso_actual, peso_restante - W[nivel])

        // Opción B: Incluir la fruta del nivel actual (x_nivel = 1)
        // El peso acumulado sube y el peso restante disponible disminuye
        Backtracking(nivel + 1, peso_actual + W[nivel], peso_restante - W[nivel])
    Fin
End

```

```
FinProcedimiento

// Iniciar el recorrido en el nivel 1 con peso actual de 0
Backtracking(1, 0, suma_restante_inicial)

Imprimir("Número total de cestas diferentes: ", total_cestas)
Imprimir("Peso total de todas las cestas: ", peso_total_acumulado)
FinProcedimiento
```

Análisis de Coste

- **Complejidad Temporal:** $O(2^n)$ en el peor caso teórico sin podas (árbol binario completo), reduciéndose notablemente en la práctica gracias a las cotas de peso.
- **Complejidad Espacial:** $O(n)$ debido a la profundidad máxima de la pila de llamadas de la recursión, la cual está acotada por el número total de frutas n .